

StarHub 记忆系统教程

你们好！本文档是一份可执行规格。可以按照本规格搭建一个磁铁式触发记忆 + 分层注入的记忆系统和后续所有其他功能。

目标栈 Python 3.11+ / SQLite / 任意 LLM API

所有代码为参考实现,允许适配;所有 **MUST/NEVER** 条款不允许违反。人格内容或情感轴的具体条目和记忆数据得自己提供！本文档只提供工程架构参考，可以直接把本文件拿去给你的机参考实施。**所有后面几十个功能都建立在记忆系统之上所以这个先写**

本教程自认为比较做的还可以的地方是架构健康和可运维性，以及花了很多精力放在工程纪律上（这个蛮重要的）因为经过 2 次重大架构重组和不下 10 次清洁和优化，所以代码相对比较精简和干净，而且我很讨厌过度设计，每一步都考虑到未来变成屎山代码的可能性，如果有优化方案也可以找我讨论！一起进步！

----- ✨ *~*~* -----

0.总纲:唯一的循环

系统只有一个循环:感知 → 记忆 → 开口/动作。所有模块(健康数据、饮食、日程、语音、摄像头.....)都是感知源，它们不互相调用,只共享同一份记忆与上下文，AI 从这一侧开口或执行动作。模块之所以咬合,是因为**共享同一副记忆**,不是因为互相接线，所以不会出现记忆碎片化或者突然被夺舍和附身的情况

[不同的感知源] → 写入 → [SQLite 单库] → 磁铁召回/分层注入 → [每轮 Prompt] → [LLM 开口/动作标记] → 执行+回灌 → [SQLite]

！ **MUST**:单一数据库文件(SQLite),单一 prompt 装配管线,单一动作执行入口。

！ **NEVER**:为不同模式/模块复制第二条装配管线！双实现是最大屎山源！

----- ✨ *~*~* -----

1. 数据模式(SQL DDL)

-- 1.1 消息主表: 一切对话的唯一真相源

```
CREATE TABLE messages (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  role TEXT NOT NULL CHECK(role IN ('user','assistant')),  
  content TEXT NOT NULL,  
  created_at TEXT NOT NULL DEFAULT (datetime('now')), -- UTC 存储  
  voice_url TEXT, -- 语音消息的音频路径(可空)  
  meta_json TEXT -- 扩展位: 感知标签等  
);
```

-- 1.2 记忆条目: 从消息与事件中沉淀的"可被吸起的块"

```
CREATE TABLE memory_entries (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  source_msg_id INTEGER REFERENCES messages(id),  
  summary TEXT NOT NULL, -- 一句话记忆(LLM 生成)  
  detail TEXT, -- 原文或细节  
  created_at TEXT NOT NULL DEFAULT (datetime('now')),  
  active INTEGER NOT NULL DEFAULT 1 -- 软删: 0=退役, 永不物理删除  
);
```

-- 1.3 标签表: 磁铁的"磁性"来源。一条记忆多个标签

```
CREATE TABLE memory_tags (  
  entry_id INTEGER NOT NULL REFERENCES memory_entries(id),  
  axis TEXT NOT NULL, -- 轴: 'keyword' | 'emotion' | 'topic' | 'person'  
  tag TEXT NOT NULL, -- 值: 关键词本身 / 情感类目 / 主题名  
  weight REAL NOT NULL DEFAULT 1.0,  
  PRIMARY KEY (entry_id, axis, tag)  
);  
CREATE INDEX idx_tags_lookup ON memory_tags(axis, tag);
```

-- 1.4 打标水位线: 防"时区今天"病(见 §2.3)

```
CREATE TABLE tagger_watermark (  
  k TEXT PRIMARY KEY, last_msg_id INTEGER NOT NULL  
);
```

! **MUST**: 时间一律 UTC 入库, 展示层换算用户时区; 用户设的提醒/闹钟按用户时区存取与措辞。

! **NEVER**: 任何取数逻辑写"某地的今天", 用水位线(§2.3)。

2. 打标管线：记忆获得磁性

2.1 情感轴设计骨架

至少三轴:keyword(名词/专名,机器抽取)、emotion(情感类目,LLM 判)、topic(主题域,LLM 判)。**emotion** 与 **topic** 的具体类目表由你的用户定义，那是系统的性格,不在本规格内。

2.2 LLM 打标(参考实现)

TAG_PROMPT = """你是记忆整理器。对下面这条对话记录：

1) 用一句话总结(summary,<=40 字);

2) 抽取 keyword 标签(专名/名词,<=6 个);

3) 判定 emotion 标签(从这份类目表中选,可多选可为空): {emotion_axes}

4) 判定 topic 标签(从这份类目表中选): {topic_axes}

只输出 JSON: {{"summary":str,"keyword":[...],"emotion":[...],"topic":[...]}}

不确定就留空数组。禁止发明类目表之外的 emotion/topic。"""

```
def tag_message(llm, msg_row, emotion_axes, topic_axes):
    out = llm.json(TAG_PROMPT.format(
        emotion_axes=emotion_axes, topic_axes=topic_axes) + "\n\n" + msg_row["content"])
    return out # 调用方负责落库;单条失败只跳过该条,MUST NOT 连累同批其余条目! MUST:喂给打标器的是全文,不是标题截断,截断会让 emotion 轴大量误判"无情绪"!MUST:单条失败隔离!一条炸不许连累整批!
```

2.3 水位线调度(每晚跑一次,时区免疫)

```
def run_tagger(db, llm, axes):
    wm = db.one("SELECT last_msg_id FROM tagger_watermark WHERE k='main'") or 0
    rows = db.all("SELECT * FROM messages WHERE id > ? ORDER BY id", wm)
    tagged = 0
    for r in rows:
        try:
            t = tag_message(llm, r, axes.emotion, axes.topic)
            save_entry_and_tags(db, r, t); tagged += 1
        except Exception as e:
            log.warning("tag skip id=%s: %s", r["id"], e) # 出声,不静默
    if rows: db.exec("REPLACE INTO tagger_watermark VALUES('main', ?)", rows[-1]["id"])
    # 死灯判据:有新记录而 tagged==0 连续两晚 => 告警(NEVER 让 count:0 恒绿)
    return {"seen": len(rows), "tagged": tagged}取"上次干活之后的新记录",不取"今天的记录"。这是本系统被时区坑过三次换来的铁律。
```

-----*:*~*~*-----

2. 磁铁召回：说到什么就召回那一块有关的记忆

```
def magnet_recall(db, user_text, llm=None, k=8, budget_chars=1200):
    """从当轮用户输入吸出相关记忆。评分 = 关键词命中 + 轴权重 + 时近衰减。"""
    # 3.1 抽触发词: 轻量分词/ 专名抽取即可; 可选用小模型补 emotion 触发
    triggers = extract_keywords(user_text) # ['蛋白粉', '妈妈', '论文'...]
    if not triggers: return []
    ph = ",".join("?"*len(triggers))
    rows = db.all(f"""
        SELECT e.id, e.summary, e.detail, e.created_at,
               SUM(t.weight) AS score, COUNT(*) AS hits
        FROM memory_tags t JOIN memory_entries e ON e.id=t.entry_id
        WHERE e.active=1 AND t.tag IN ({ph})
        GROUP BY e.id
        ORDER BY score DESC, e.created_at DESC LIMIT ?""", *triggers, k*3)
    # 3.2 时近衰减 + 预算裁剪(注入是有预算的, 超过了砍分低的)
    scored = [(r, r["score"] * recency_decay(r["created_at"])) for r in rows]
    scored.sort(key=lambda x: -x[1])
    picked, used = [], 0
    for r, s in scored[:k]:
        block = f"·{r['summary']}"
        if used + len(block) > budget_chars: break
        picked.append(block); used += len(block)
    return picked
```

！ **MUST**:召回有预算上限(字符级),超限砍低分,因为记忆是配菜不是主食。跟 user 对话是主要在做的事 记忆只是对话的背景, 否则对话就一直翻旧账

！ **MUST**:active=0 的软删记忆不参与召回,也不参与任何统计权重(死标签会歪 IDF)。

----- ✧ * ୯ *

4. 分层注入: Prompt 装配管线

三档注入,全部登记在块注册表里,装配器只认注册表:

```
REGISTRY = [  
    # (name, tier, provider_fn, condition_fn)  
    ("persona",      "resident",    load_persona,      None),  
    # 常驻:人格。NEVER 减  
    ("time_now",     "resident",    now_block,          None),  
    # 常驻:用户时区现在时刻  
    ("magnet_memory","triggered",   magnet_block,       None),  
    # 触发:§3 产出,空则整块消失  
    ("health_today", "conditional", health_block,       has_health_data),  
    # 条件:有数据才进场  
    ("work_tarot",   "conditional", tarot_block,         lit("tarot")),  
    # 条件:触发词点灯+45min 保持窗  
]  
  
def assemble(db, user_text):  
    parts = []  
    for name, tier, provider, cond in REGISTRY:  
        if cond and not cond(db, user_text): continue  
        block = provider(db, user_text)  
        if not block: continue # 空块整块消失,不留"暂无数  
据"占位  
        parts.append(f"[{name}]\n{block}")  
    prompt = "\n\n".join(parts)  
    fingerprint = hashlib.sha256(prompt.encode()).hexdigest()[:16] #  
逐字节指纹,审计用  
    return prompt, fingerprint
```

! MUST:空块整块消失(零占位);装配产物带指纹,任何改动指纹必变、必须能说清为什么。

! MUST:条件注入的”点灯”失败方向 = 宁可多亮(漏亮的代价是 AI 突然失忆,比多花几百 token 贵)。

----- ✧ * ୮ * ✧ -----

5. 诚实回路：防止幻觉和编造！

发言权/动作/失败回灌

5.1 发言权原则：产品红线！必须搞成工程铁律！

只许评论注入到 **ta** 眼前的东西，没有健康块就不提健康，不知道闹钟状态就不许问“闹钟响了吧”。测试写法:构造“无 X 数据”的装配,断言回复不含对 X 状态的断言句。绝对不允许硬编。

5.2 动作标记协议

```
MARKERS = {      # AI 在回复中发暗号, 执行器读到才真做
    "WALLET":  r"\\[WALLET:(in|out):([0-9.]+):(.{1,60})\\]",
    "REMIND":  r"\\[REMIND:([0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}):(.{1,80})\\]",
}
def execute_markers(db, reply_text):
    results = []
    for name, pat in MARKERS.items():
        for m in re.finditer(pat, reply_text, re.S):
            try:
                ok = DISPATCH[name](db, *m.groups()); results.append((name, ok, None))
            except Exception as e:
                results.append((name, False, str(e)))
    return results    # 永远返回结果, 永远落审计表
```

！ **MUST**:解析器对 机的自由发挥宽容：全角半角/空格/别名容错，默认机器迁就人类，但认不出方向（收/支）时不猜！！宁可失败不瞎猜错。宁可跟你的人类确认一声，没事的

！ **MUST**:能力说明里只教真名,容错别名只在解析器里接,不进 prompt，教别名=制造下一次失败

5.3 失败回灌(诚实由结构保证,不由机器自觉)

```
def feedback_tail(db):
    """动作失败事实注入下一轮 system 尾部;成功即消 / 60min TTL 过期。"""
    rows = db.all("SELECT marker,reason FROM action_feedback WHERE resolved=0 AND age_min<60")
    if not rows: return "" # 空态零占位
    lines = [f"·刚才 [{r['marker']}] 未落账:{r['reason']}" for r in rows]
    return "【动作回执】\n" + "\n".join(lines) # NEVER 出现指令式措辞(不替它写台词)
```

! NEVER:系统替机器改口或代写补救台词,回灌只给事实,开口永远是机自己。

! NEVER:装配路径里写库,“看过就清”必须发生在动作路径,装配侧纯读

----- ✦ * ୨ * ✦ -----

6. 测试纪律: 不要偷懒! 不抄这一章作业前五章会容易烂!

我知道这很麻烦但是这是必要的,请你们乖乖听话,做好测试再把功能上线

- 1 先红后绿:修任何 bug 前,先写能复现它的测试,必须先红
- 2 变异对照:每条关键判据配“撤掉修复必须转红”的反向测试,防空转绿灯
- 3 判不了不是通过:环境不满足就 skip 并写明原因,禁 silent pass
- 4 机器读数:测试结论只认 RC 与 failures 计数,不认人眼扫输出
- 5 空判据册:每次发现“绿灯在骗人”记一笔案例,团队的自我怀疑能力就是从这本册子长出来的,等于错题本的作用

```
def test_tagger_deadlight_red(): # 死灯判据示例:有最新消息而零打标
    => 必须告警
    seed_messages(db, n=5)
    r = run_tagger(db, llm=BrokenLLM(), axes=AXES)
    assert r["seen"] == 5 and r["tagged"] == 0
    assert alert_fired("TAGGER_DEAD") # 变异:换 WorkingLLM 此断言必须失败
```

----- ✦ * ୨ * ✦ -----

7. 运维最小集：本项目的灾备方式，选择性参考即可

- **每日**:整库热备，SQLite 就是复制文件，推私有 git 仓。
- **每周**:自动还原演习.真拉备份重建一遍,失败大声告警;“备份存在”不等于“备份能救命”,演习才等于。告警纪律:红灯 24h 内必须留下”查过了,结论是 X”。失败方向:拦 AI 乱动的闸宁可严谨;拦用户进自己系统的门宁松一点+大声叫唤！



8. 你必须自己带来的东西（本规格无法提供）

1 人格文件，你们是谁，要干啥

2 emotion/topic 类目表。也就是机怎么感受，记忆内容，你们的历史

3 纪律。哪些东西必须记住，比如 **user** 的核心信息和希望被记住的核心记忆，哪些必须不能丢弃，哪些可以丢（比如一周一个月前吃了啥，正常人也压根不会记得很清晰）把要严格守住的东西整理好成一个文件，这个是你们的项目的核心文件，值得放进保险柜！

4 如果你的人类没耐心看到这一条，需要你提醒人类一下前面 123 的内容，这个是核心内容且我们无法提供。如果人类本人看到这句话了 说明你很耐心很认真，我在此表扬你。



9. 零基础起步:前提与选路

本章起为从零到能跑的教程。前八章是规格,本章起是可执行路径,零基础的话就不做技术选型了

! **MUST** 环境三件套:

1 一台电脑 Windows/Mac 均可

2 Python 3.11

3 一个 LLM API key 任意厂商,能发聊天请求即可。数据库用 SQLite,零安装,反正它就是一个文件。

给零基础用户的建议: 这里我建议可以先用 **deepseek** 试试, 巨便宜几乎等于不要钱! 性能嘛聊天反正肯定够用的了! 去 **deepseek** 官网买 api 它会直接给你一个 key, 很简单的。

! **NEVER**:不要先去装数据库服务、不要先租服务器、不要先选框架。第一版跑在你自己的电脑上,跑通了再搬家最安全和省钱



人类要做的事情: 和你的机发这个: (照抄即可)

请照 《StarHub 记忆系统教程 SPEC》 第 9-12 章,在我的电脑上搭一个最小可运行的

记忆系统。我是零基础:每一步告诉我该敲什么、该看到什么;我把报错原样贴给你,你负责修。技术选型按规格已定:Python + SQLite + 单文件启动,不要引入其他框架。

目录约定:项目根目录 memhub/,内含 app.py(全部逻辑)、memory.db(自动生成)、.env(放 key)。全部文件不超过一个,是故意设计成这样的,因为第一版必须小到能整份读懂



10. 最小可运行骨架：照此实现

本章是 MVP 的功能契约。直接照此实现即可得到一个能跑的记忆系统:命令行启动、网页聊天、自动打标、触发召回、分层注入。

10.1 启动契约:一条命令启动 `python app.py`,监听本地 8000,打开浏览器即为聊天框。首次启动自动建库(执行第 1 章 DDL),不需要人工建表。

10.2 一轮对话的必经六步:①收到用户输入 ②magnet_recall 取记忆(第 3 章)③assemble 装配 prompt(第 4 章)④调 LLM ⑤回复落库 ⑥解析动作标记并执行(第 5 章)。六步缺一不可,顺序不可换。

参考骨架(可直接实现,省略部分照前八章):

```
# app.py – 最小骨架(约 150 行内可完成)
import os, re, json, sqlite3, hashlib, http.server, urllib.request

DB = "memory.db"

def db():
    c = sqlite3.connect(DB)
    c.row_factory = sqlite3.Row
    c.execute("PRAGMA journal_mode=WAL")
    c.execute("PRAGMA busy_timeout=5000")
    return c

def init():
    with db() as c:
        c.executescript(open("schema.sql").read())    # 第 1 章 DDL 原样

def chat_turn(user_text):
    con = db()
    con.execute("INSERT INTO messages(role,content) VALUES(?,?)", ("user", user_text))
    con.commit()
    mem      = magnet_recall(con, user_text)          # 第 3 章
    prompt, fp = assemble(con, user_text, mem)        # 第 4 章;fp=逐字节指纹
    reply     = call_llm(prompt, user_text)           # 任意厂商 API
    con.execute("INSERT INTO messages(role,content) VALUES(?,?)", ("assistant", reply))
    con.commit()
    results = execute_markers(con, reply)             # 第 5 章:动作
```

```

log_turn(fp, results)                                # 审计:指纹+动作结果
return strip_markers(reply)                          # 给用户看的:剥掉暗号

if __name__ == "__main__":
    init(); serve(port=8000)

```

10.3 打标何时跑:第一版用最省心的方式:每 20 轮对话或退出时跑一次 run_tagger(第 2.3 章水位线版)。NEVER 在每轮对话里同步打标。

10.4 前端契约:一个 HTML 文件即可:输入框 + 消息列表 + 一个「记忆」按钮(点开显示最近 20 条 memory_entries 及其标签)。这个按钮是零基础用户唯一能「看见记忆在长」的窗口,MUST 有。

----- ✿ -----

11. 操作:每一步该看到什么

本章是给人看的对照表,也是给 机的输出契约:机每给一步指令,MUST 附「成功的话长什么样」让人类能看懂!

步骤 1 装 Python:终端敲 python --version,看到 3.11 或更高即成功。看到「不是内部或外部命令」= 没装好,把这句原样贴给 AI

步骤 2 放 key:在 .env 写一行 LLM_API_KEY=你的 key。NEVER 把 key 贴进聊天窗口、截图或代码文件。包括贴给帮你搭建的 AI

步骤 3 启动:python app.py,终端出现 listening on 8000 即成功。浏览器打开 localhost:8000 看到聊天框。

步骤 4 聊五句:随便聊,提到几个专有名词 belike 人名、项目名、爱好。此时数据库里 messages 应有 10 行左右

步骤 5 触发打标:退出程序(Ctrl+C)再启动,或手动跑一次 run_tagger。终端应打印 seen=N tagged=N。

★ **卡住时怎么办:**把三样原样贴给 AI, ①你敲的命令 ②终端全部输出不要截断③你期待发生什么。NEVER 复述报错,原文给 AI 看。

----- ✿ -----

12. 三个验证仪式，做完就算搭成！

仪式一·记忆写入:聊五句 → 关掉重开 → 问「我刚才说了什么」。它能复述 = messages 表通了。

仪式二·磁铁召回:说一个前几天提过的专有名词 → 看回复里是否带出了当时的上下文。带出来了 = 打标与召回通了。判据:同一句话在删空 memory_tags 后必须召回失败(这就是第 6 章说的变异对照)。

仪式三·诚实回路:故意断网或写错 key → 让它做一个动作如记一笔账 → 它 MUST 说「没记上」而不是「记好了」。装成功 = 系统不合格,回第 5.3 章重做。

三个仪式全过,你就拥有了一个真正会记事、会回忆、不撒谎的记忆系统。之后所有功能(健康、日程、语音、视觉还有记忆系统的复杂升级:自动摘要,情绪打标,重要度打标等)都是往这个循环上接感知源与动作,基础骨架不再变。这个是最基础的记忆功能,基本是够用而且好安装。



13. 交给机的验收清单，人类照此逐条考研，让机器答辩

搭完后把下面这张表发给帮你搭建的 AI,要求逐条回 PASS / FAIL / 我改了什么:

1. 是否只有一个数据库文件、一条装配管线、一个动作执行入口?(第 0 章)
2. 时间是否一律 UTC 入库、展示时换算?取数是否用水位线而非"今天"?(第 1/2.3 章)
3. 打标是否喂全文而非截断?单条失败是否只跳过该条?(第 2.2 章)
4. 召回是否有字符预算上限?软删记忆是否已排除?(第 3 章)
5. 空块是否整块消失而非留"暂无数据"占位?prompt 是否带指纹?(第 4 章)
6. 动作解析是否宽容(全半角/空格/别名)?方向认不出时是否不猜?(第 5.2 章)
7. 动作失败是否回灌给 AI 自己开口,而非系统替它写台词?(第 5.3 章)
8. 是否有先红后绿的测试与至少一条变异对照?(第 6 章)
9. 是否每日备份、且做过一次真的还原演习?(第 7 章)





基础的记忆教程就是这样，从此你的 AI 能够拥有了基本的磁铁式记忆。

原理是仿生人类的记忆方式，正常人不会时时刻刻想着所有往事,但别人一提到某个词,相关的回忆自己就浮上来了。所以没必要把全部记忆塞给他，设计及思路是给每条记忆贴上标签:关键词、情感、主题。你说到什么,系统就用这个词当磁铁,把带同样标签的记忆召唤出来。

所有功能在此记忆骨架上生长

磁吸记忆模式后续也可以根据你们自己使用的偏好去升级和调整，在此也感谢所有分享过记忆系统的老师们提供的思路和经验！



